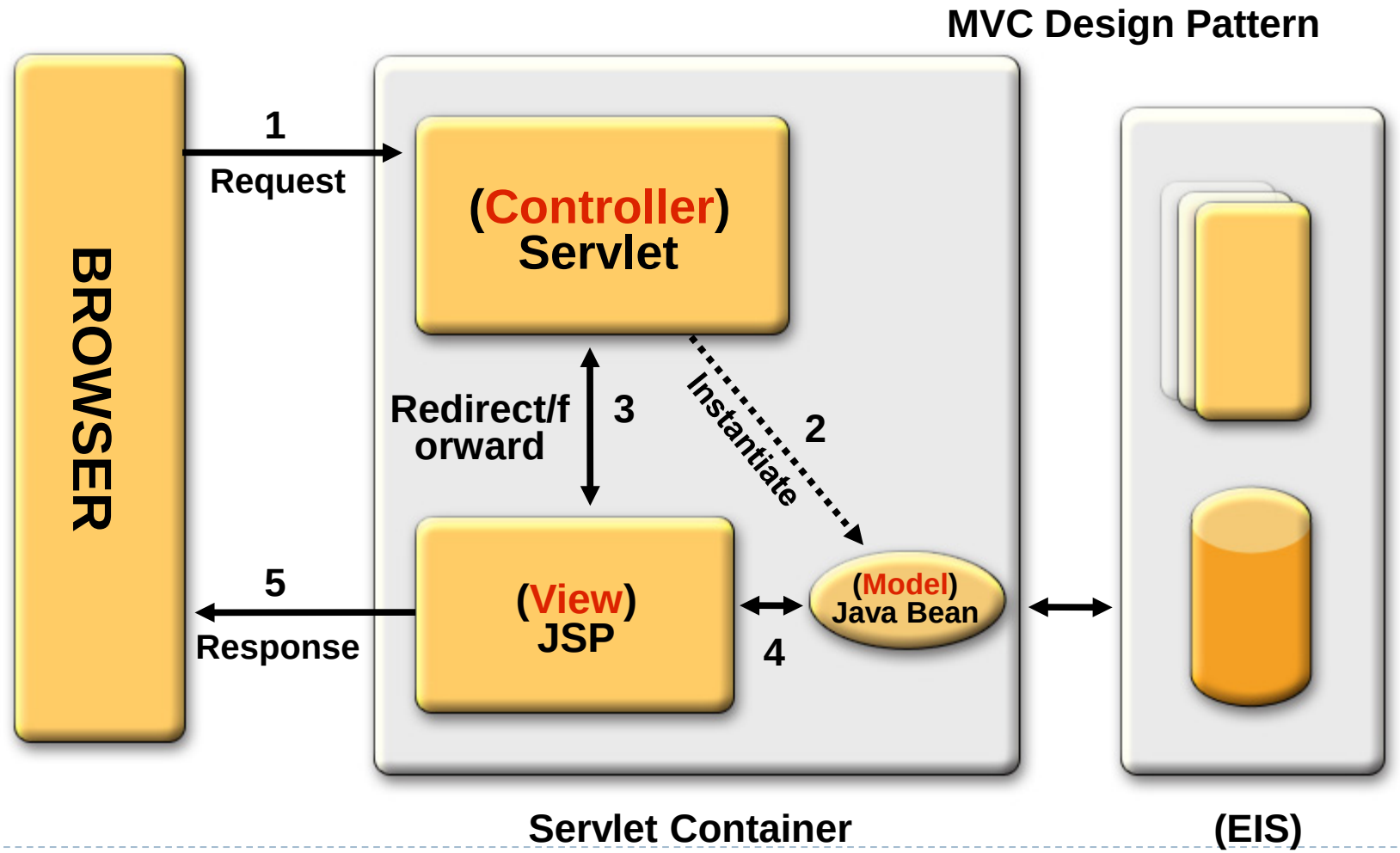
Servlet, JSP, Java Bean, JDBC MVC Servlet Centric Model

Learning Objectives

- To work with Servlet, JSP, Java Bean and JDBC together.
- To build a simple CRUD application (Servlet-centric)
  - To show all customer records
  - To add a new customer record
  - To search customer record
  - To delete a customer record
  - To edit existing customer record
- To understand model 2 MVC Architecture
-

Model 2 Architecture (Servlet-centric)

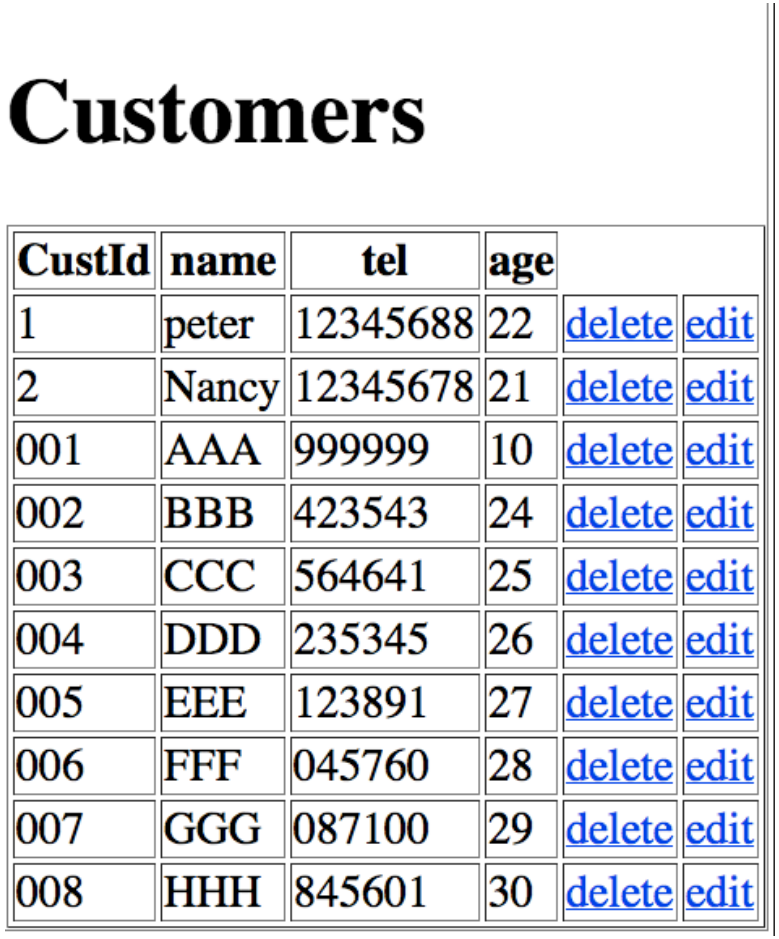


Demo

welcome.jsp


Linked to editCustomer.jsp
Welcome to the ICT
add Customer
list Customer
handleCustomer?action=list
Search customer:
submit
Logout

handleCustomer?action=list



Customers

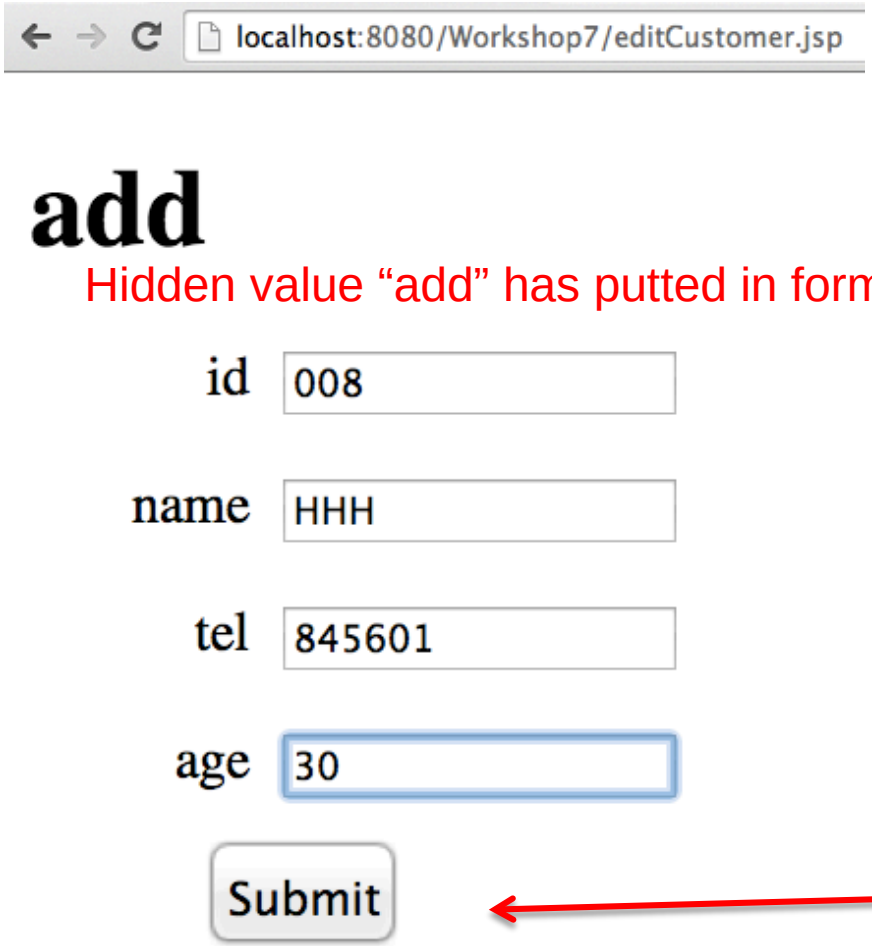
CustId	name	tel	age		
1	peter	12345688	22	delete	edit
2	Nancy	12345678	21	delete	edit
001	AAA	999999	10	delete	edit
002	BBB	423543	24	delete	edit
003	CCC	564641	25	delete	edit
004	DDD	235345	26	delete	edit
005	EEE	123891	27	delete	edit
006	FFF	045760	28	delete	edit
007	GGG	087100	29	delete	edit
008	HHH	845601	30	delete	edit

List Customer hyper link is clicked and

“handleCustomer?action=list” is requested

Add a customer

1. HandleCustomer Servlet handles "add" action
2. Redirect the result to "list" action to display all customer



Customers

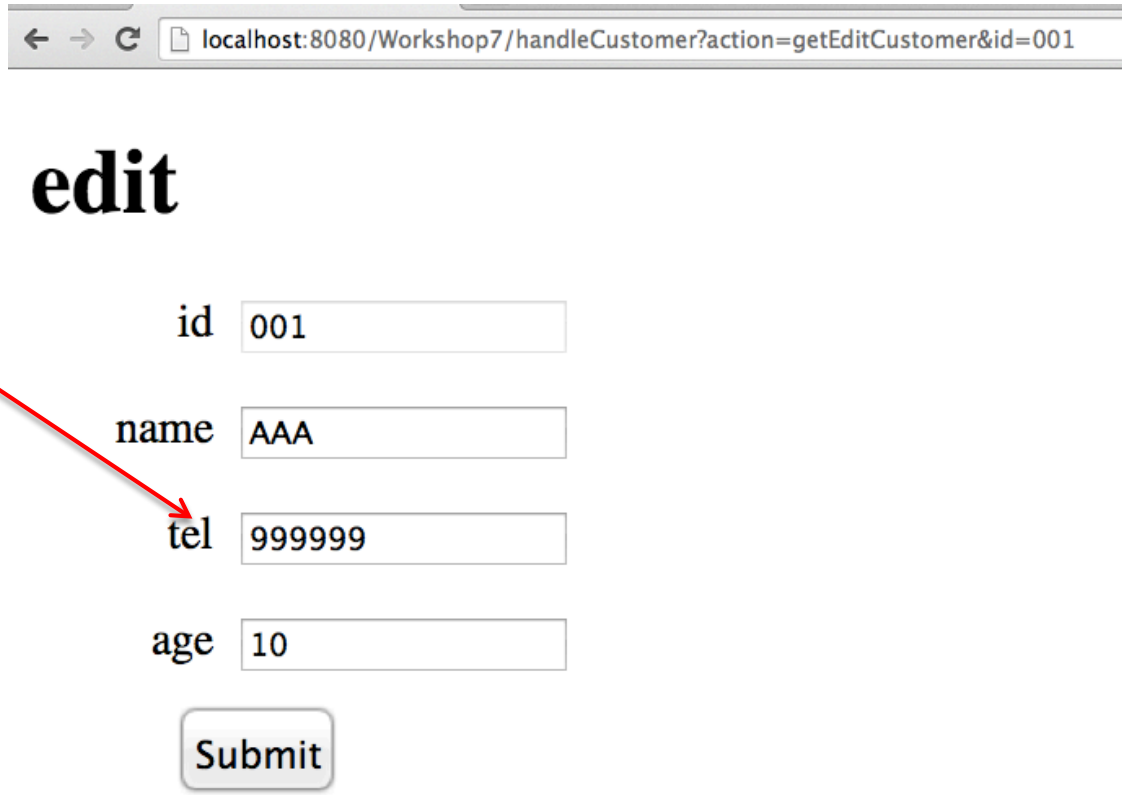
CustId	name	tel	age		
1	peter	12345688	22	delete	edit
2	Nancy	12345678	21	delete	edit
001	AAA	999999	10	delete	edit
002	BBB	423543	24	delete	edit
003	CCC	564641	25	delete	edit
004	DDD	235345	26	delete	edit
005	EEE	123891	27	delete	edit
006	FFF	045760	28	delete	edit
007	GGG	087100	29	delete	edit
008	HHH	845601	30	delete	edit

Edit Customer

1. HandleCustomer Servlet handles "getEditCustomer" action
2. Retrieve the customer from database with given id
3. Forward the result to editCustomer.jsp and show the value in the form

Customers

CustId	name	tel	age		
1	peter	12345688	22	delete	edit
2	Nancy	12345678	21	delete	edit
001	AAA	999999	10	delete	edit
002	BBB	423543	24	delete	edit
003	CCC	564641	25	delete	edit
004	DDD	235345	26	delete	edit
005	EEE	123891	27	delete	edit
006	FFF	045760	28	delete	edit
007	GGG	087100	29	delete	edit
008	HHH	845601	30	delete	edit



1. EditCustomer handles “edit” action
2. Update the customer record in database
3. Redirect the result to “list” action for showing all customers

Customers

CustId	name	tel	age		
1	peter	12345688	22	delete	edit
2	Nancy	12345678	21	delete	edit
001	AAA	999999	10	delete	edit
002	BBB	423543	24	delete	edit
003	CCC	564641	25	delete	edit
004	DDD	235345	26	delete	edit
005	EEE	123891	27	delete	edit
006	FFF	045760	28	delete	edit
007	GGG	087100	29	delete	edit
008	HHH	845601	30	delete	edit

Search Customer

action “search” is defined as hidden value

Search customer:

A

submit

All the customer whose name contains “A” is found

Customers

CustId	name	tel	age		
001	AAA	999999	10	delete	edit
009	Alice	123456	16	delete	edit
2	Nancy	12345678	21	delete	edit

After completing in Workshop 5, you should have

- ict.bean.CustomerBean.java, ict.db.CustomerDB.java and a few test cases
- ict.db.CustomerDB java contains the following method
  - queryCust() which returns an ArrayList of customer bean object.
  - queryCustByID(String id) which queries the customer record by custID. It returns a customer bean object.
  - queryCustByName(String name) which queries the customer record by name. It returns an array of customer bean object.
  - The method delRecord(String custId) which deletes a record with a particular CustId.
  - The method editRecord(String custId, String relevantPropeties) which updates the values of a record except its CustId.

Prerequisites

1) Database setup (Refer to workshop 5 for details)
2) Run the test cases to
    1) create the customer table
    2) add a few customer records

Create a Web Application Project

- Project Name WS7_YourStduent NO
- Copy the following files your project
  - ict.bean.CustomerBean.java
  - ict.db.CustomerDB.java
  - a few test cases

Task 1
List all Customer's Record

To display all customer records in the listCustomers.jsp

- Create a HandleCustomer servlet to handle customer actions.
  - Define the HandleCustomer under package ict.servlet
  - It handles the action delete, search and list actions
- Create a listCustomers.jsp to show all customer records

Create a ict.servlet.**HandleCustomer** Servlet

```
public class HandleCustomer extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

  protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
}
```

Put processRequest Method tot **HandleCustomer** Servlet

```
protected void processRequest(HttpServletRequest request,
HttpServletResponse response)  throws ServletException, IOException {

String action = request.getParameter("action");

      if ("list".equalsIgnoreCase(action)) {
         // call the query db to get retrieve for all customer
          ArrayList<CustomerBean> customers = db.queryCust();
            // set the result into the attribute
          request.setAttribute("customers", customers);
            // redirect the result to the listCustomers.jsp
          RequestDispatcher rd;
          rd = getServletContext().getRequestDispatcher("/listCustomers.jsp");
          rd.forward(request, response);
      } else {
          PrintWriter out = response.getWriter();
          out.println("No such action!!!");
      }
  }
```

CustomerDB and Init Method in **HandleCustomer**Servlet

```
private CustomerDB db;

public void init() {
//1.  obtain the context-param, dbUser, dbPassword and dbUrl which defined in web.xml
//2.  create a new db object  with the parameter
}
```

Init method to initialize the database object

```
String dbUser = this.getServletContext().getInitParameter("dbUser");
String dbPassword = this.getServletContext().getInitParameter("dbPassword");
String dbUrl = this.getServletContext().getInitParameter("dbUrl");

    db = new CustomerDB(dbUrl, dbUser, dbPassword);
```

Make sure you have put the Context param in Web.xml

```
<context-param>
    <param-name>name</param-name>
    <param-value>value</param-value>
</context-param>
```

```
<context-param>
    <param-name>dbUrl</param-name>
    <param-value>jdbc:mysql://localhost:8889/ITP4511_DB</param-value>
</context-param>
<context-param>
    <param-name>dbUser</param-name>
    <param-value>root</param-value>
</context-param>
<context-param>
    <param-name>dbPassword</param-name>
    <param-value>root</param-value>
</context-param>
```

Make sure you have done the servlet mapping

- Map the action, /handleCustomer, to ict.servlet.HandleCustomer

- @WebServlet(urlPatterns= ……)
- public class HandleCustomer…….

In the Customer Servlet, an ArrayList of customer has already stored as attribute in the request
request.setAttribute("customers", customers);

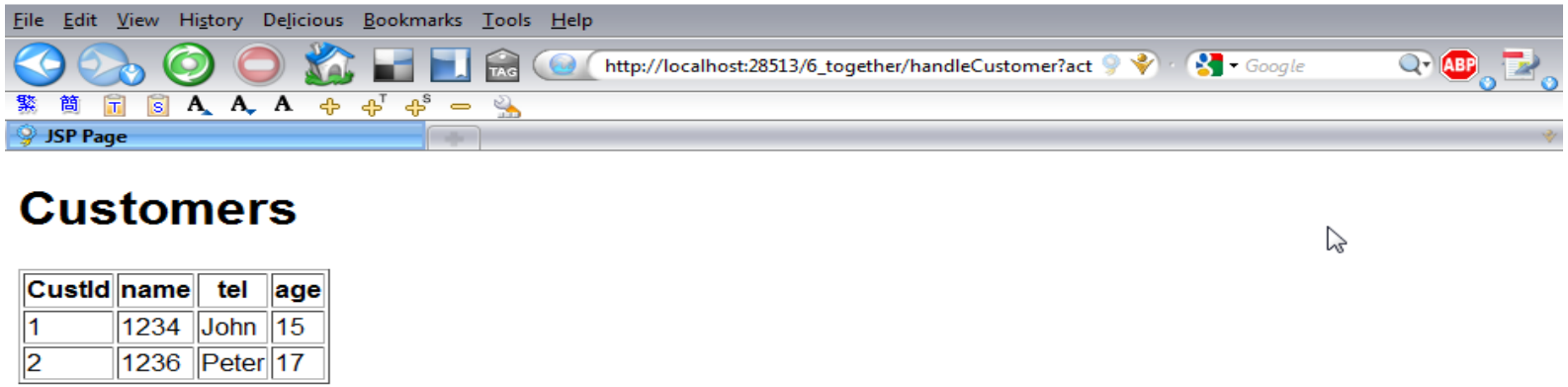
Create listCustomers.jsp

```
<%
        ArrayList<CustomerBean> customers =
(ArrayList<CustomerBean> )request.getAttribute("customers");
        out.println("<h1>Customers</h1>");
        out.println("<table border='1'          >");
        out.println("<tr>");
        out.println("<th>CustId</th>  <th> name</th><th> tel</th><th> age</th >");
        out.println("</tr>");
      // loop through the customer array to display each customer record
        for (int i = 0; i < customers.size(); i++) {
            CustomerBean c = customers.get(i);
            out.println("<tr>");

            out.println("<td>" + c.getCustId() + "</td>");
            out.println("<td>" + c.getName() + "</td>");
            out.println("<td>" + c.getTel() + "</td>");
            out.println("<td>" + c.getAge() + "</td>");
            out.println("</tr>");

        }
        out.println("</table>");
%>
```

Show the customers records By handleCustomer?action=list



Customers

CustId	name	tel	age
1	1234	John	15
2	1236	Peter	17

Make sure you have set mysql lib

ADD a new Customer

Define a customer input form (editCustomer.jsp)

- The form is specified customer properties
  - id, name, tel and age
- When submit is click, a request is made to "handleEdit" which is mapped to ict.servlet.HandleEdit.java
- A hidden "action" value "add" is submitted as parameter.

```
<form  method="get" action="handleEdit">
        <input type="hidden" name="action"  value="add" />
        ID  <input name="id"  type="text" value=""/> <br>
        Name <input name="name"  type="text" value=""/> <br>
        Tel <input name="tel"  type="text" value=""/> <br>
        Age <input name="age"  type="text" value=""/> <br>
        <td ><input type="submit" value="submit"/> <br>
  </form>
```

Create a ict.servlet.**HandleEdit** Servlet

- The servlet handles the requests from the editCustomer.jsp
- It handles "add" and "edit" action
- Define the servlet under package ict.servlet

handleEdit Servlet

```
//1.  define db object
//2.  define init method
//2.1 doGet method, in which call processRequest
//2.2 doPost method, in which call processRequest
protected void processRequest(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

       // 3.  get the parameter, action, from users
       // 4. if action equals to “add”
       // 4.1 call  the database operations  db.addRecord(id, name, tel, ag
       // redirect the result
        response.sendRedirect("handleCustomer?action=list");
      } else {
            PrintWriter out = response.getWriter();
            out.println("No such action!!!");
        }
   }
```

- 1 List Customer

Customers

CustId	name	tel	age
1	Peter	12345688	21
2	Nancy	12345678	21

- 2 Enter info and submit

id 3

name Amy

tel 2345

age 18

- 3. Result

Customers

CustId	name	tel	age
1	Peter	12345688	21
2	Nancy	12345678	21
3	Amy	2345	18

Complete the following tasks

- Delete a customer record by specifying the id
  - handleCustomer?action=delete&id=2
- Search a customer records and display the result
  - a search customer input form
  - handleCustomer?action=search&name=peter
- Edit a customer record using with an input form

For edit and delete actions

- add two columns in listCustomers.jsp

```
// loop through the customer array to display each customer record
for (int i = 0; i < customers.size(); i++) {
    CustomerBean c = customers.get(i);
    out.println("<tr>");

    out.println("<td>" + c.getCustid() + "</td>");
    out.println("<td>" + c.getName() + "</td>");
    out.println("<td>" + c.getTel() + "</td>");
    out.println("<td>" + c.getAge() + "</td>");
    out.println("<td><a href=\"handleCustomer?action=delete&id=" + c.getCustid() + "\">delete</a></td>");
    out.println("<td><a href=\"handleCustomer?action=getEditCustomer&id=" + c.getCustid() + "\">edit</a></td>

    out.println("</tr>");

}
out.println("</table>");
```

listCustomers is shown after two columns are added (edit and delete actions)

CustId	name	tel	age		
007	GGG	087100	11	delete	edit
008	HHH	845601	30	delete	edit
abc	1	1	12	delete	edit
1111	abcs	12345	10	delete	edit
33333	1a	2	123	delete	edit
001	AAA	999999	10	delete	edit
002	BBB	423543	24	delete	edit
1	John	123	3	delete	edit
21	jesse	123	12	delete	edit
2	2	2	333	delete	edit

delete record with the given id

handleCustomer?action=delete&id=007

edit record with the given id

handleCustomer?
action=getEditCustomer&id=008

Update handleCustomer for delete action

```
 if ("list".equalsIgnoreCase(action)) {
….
} else if ("delete".equalsIgnoreCase(action)) {
            // get parameter, id, from the request
          if (id != null) {
               // call delete record method in the database

               // redirect the result to list action
            }
}
```

Try the delete action

Update handleCustomer for getEditCustomer action

```
else if ("getEditCustomer".equalsIgnoreCase(action)) {

        // obtain the parameter id;

        if (id != null) {
            // call  query db to get retrieve for a customer with the given id
          // set the customer as attribute in request scope
            request.setAttribute("c", customer);
          // forward the result to the editCustomer.jsp
        }
}
```

Update the editCustomer.jsp

- Obtain a customerBean object from request using jsp action
- Use jsp Expression to specify the value in the form with value in the customerBean
- Determine the type of action

In the servlet, customer is set as an attribute
request.setAttribute("c", customer);

```
<jsp:useBean id="c" scope="request" class="ict.bean.CustomerBean" />
<%
    String type = c.getCustid() != null ? "edit" : "add";
%>
```

- Obtain the value from customer bean
- Make sure the id is valid

```
<%
    String id=  c.getCustid() != null ? c.getCustid(): "" ;

%>
```

When “getEditCustomer” action is requested in the handleCustomer

edit

id 001

name AAA

tel 999999

age 10

Submit

Input box has shown the value of given id.

Update handleEdit for edit action

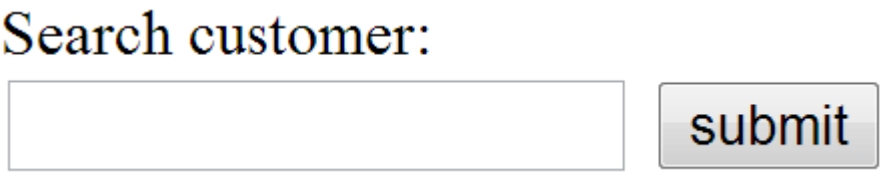
```
else if ("edit".equalsIgnoreCase(action)) {

        // obtain the parameters from request
        // call  editCustomer to update the database record
        // redirect the result to “list” action again
      response.sendRedirect("handleCustomer?action=list");
    }
}
```

Try the **edit** action

Create a search customers.jsp

```
<form  method="GET" action="handleCustomer?action=search">
    <input type="hidden" name="action" value="search"/>
    Search customer: <br/>
    <input name="name"  type="text" value=""/>
    <input type="submit" value="submit"/>
</form>
```



Update handleCustomer for search action

```
else if ("search".equalsIgnoreCase(action)) {

        // obtain the parameter name;

        if (name != null) {
        ArrayList<CustomerBean> customers;
          // call  queryByName from db
          // to retrieve for customers with the given name
         // set the result into the attribute in request
          // forward the result to the listCustoemrs.jsp
       }
}
```

Bonus Part

- If you have done a confirmation page, a bonus point will be given.
- A confirmation page is a jsp page showing the inputted value to let user to confirm the previous action. Making sure the action is correct.
- User needs to confirm input result before the database.

Forward versus Redirect

- Forward
  - a forward is performed internally by the servlet
  - the browser is completely unaware that it has taken place, so its original URL remains intact
  - any browser reload of the resulting page will simple repeat the original request, with the original URL
- Redirect
  - is a two step process, where the web application instructs the browser to fetch a second URL, which differs from the original
  - a browser reload of the second URL will not repeat the *original* request, but will rather fetch the *second* URL
  - redirect is marginally slower than a forward, since it requires two browser requests, not one objects placed in the *original* request scope are not available to the second request

Forward versus Redirect

- In general, a forward should be used if the operation can be safely repeated upon a browser reload of the resulting web page; otherwise, redirect must be used. Typically, if the operation performs an edit on the datastore, then a redirect, not a forward, is required. This is simply to avoid the possibility of inadvertently duplicating an edit to the database. More explicitly :
- for SELECT operations, use a forward
- for INSERT, UPDATE, or DELETE operations, use a redirect
- *http://www.javapractices.com/topic/TopicAction.do?Id=181*